

In-depth Explanation of ASCII, Unicode, and UTF-8

ASCII (American Standard Code for Information Interchange)

- Purpose: ASCII is one of the oldest character encoding standards, developed in the early 1960s to represent English text.
- How it works: It uses 7 bits to encode characters, allowing for 128 unique symbols. These include:
 - English uppercase and lowercase letters (A-Z, a-z)
 - Digits (0-9)
 - Basic punctuation marks (like . , ! ?)
 - Control characters (like newline \n, tab \t)
- Limitation: ASCII only covers basic English characters and a few control codes. It cannot represent characters from other languages or special symbols beyond the original 128 characters.
- Example: The letter A is represented as decimal 65, binary 01000001.

Unicode

- Purpose: Unicode is a universal character set designed to encode characters from all writing systems worldwide, including symbols, emojis, and historic scripts.
- How it works:
 - Assigns every character a unique code point -- a number written as U+XXXX (hexadecimal). For example:
 - The letter A is U+0041
 - The grinning face emoji is U+1F600
 - Unicode is not a specific encoding format but rather a standard defining these code points.
- Range: It includes over 1.1 million code points organized into 17 planes, the most commonly used being the Basic Multilingual Plane (BMP).
- Storage: To store these code points in computer memory or transmit them, Unicode needs

encoding formats such as UTF-8, UTF-16, or UTF-32.

UTF-8 (Unicode Transformation Format - 8-bit)

- Purpose: UTF-8 is the most common encoding for Unicode characters on the internet and modern software.
- How it works:
 - Variable-length encoding that uses 1 to 4 bytes per character.
 - Backward compatible with ASCII: ASCII characters are stored as single bytes identical to ASCII values.
 - For characters beyond ASCII, it uses multiple bytes with specific bit patterns to indicate their length.
- Encoding structure:
 - 1-byte: 0xxxxxxx for ASCII (7 bits)
 - 2-bytes: 110xxxxx 10xxxxxx
 - 3-bytes: 1110xxxx 10xxxxxx 10xxxxxx
 - 4-bytes: 11110xxx 10xxxxxx 10xxxxxx 10xxxxxx
- Example:
 - A (U+0041) = 01000001 (1 byte)
 - grinning face (U+1F600) = 11110000 10011111 10011000 10000000 (4 bytes)
- Advantages:
 - Efficient storage for English text
 - Supports all Unicode characters
 - Widely supported and default encoding for many platforms

Why Encoding Matters

- Interoperability: Ensures that text data can be shared and correctly interpreted across different devices and systems.

- Storage efficiency: UTF-8 uses fewer bytes for common ASCII characters while still supporting all languages.
- Globalization: Allows software to support global languages, symbols, and emojis.